Install Manual
Pi Zero 2 W

How to install, update and look after the ASL-DVS web tools on a 512 MB Raspberry Pi Zero 2 W running AllStarLink with DVSwitch.

DASHBOARD SYSMON INSTMON WIFIMON 44HELPER

FILES COVERED BY THIS EDITION

TOOL	FILE	VERSION
instmon	`instmon_v2_40_0.py`	2.40.0
Installer	`install_v6_7_0.sh`	6.7.0
Dashboard	`dashboard_pi02w_v9_4_0.py`	9.4.0-pi02w
SysMon	`sysmon_pi02w_v6_13_67_11_20261011.py`	6.13.67.11-pi02w
wifimon	`wifimon_v5_28_20261007.py`	5.28
44helper	`asl_dvs_m17_44helper_v0_0_179_20261011.py`	0.0.179

In this manual

1. **What gets installed**: the tools, their ports and what the Pi02w build changes
2. **Before you start**: what the Pi needs, the root password, free memory
3. **Step 1:** get the files and install instmon
4. **Step 2:** install the rest with the installer
5. **Step 3:** open the pages and log in
6. **Step 4:** set up wifimon's WiFi list
7. **Updating** from GitHub with instmon
8. **Memory** on a 512 MB Pi: the launcher and its off switches
9. **Troubleshooting**
10. **Removing** the tools
11. **Quick reference**
12. **Phone and its dialplan**: what the Phone tab adds to Asterisk, and where the full guide is

How to read the commands

Commands are typed on the Pi, in an SSH session or on a keyboard and screen. Lines starting with `#` are notes, not commands. Replace `<pi-address>` with the Pi's IP address or host name.

```
# a note about the next line
sudo systemctl status sysmon
```

NOTE The scripts on GitHub are comment-stripped copies. Each tool's change history is in the `changelogs/` folder of the repo.

What gets installed

Each tool is a single file. Each one runs as its own systemd service and serves its own page in the browser.

TOOL	SERVICE	PORT	WHAT IT DOES
Dashboard	`asl_dvs_dashboard`	8989	Node control. Connect ASL and EchoLink nodes; tune DMR, YSF, P25, NXDN, D-STAR, XLX and M17; favorites; Phone.
SysMon	`sysmon`	9999	System monitor and health checks for the node, DVSwitch and its config files.
instmon	`instmon`	8990	Installer and manager. Install, update, start and stop the other tools; GitHub updates; Quiet System.
wifimon	`wifimon`	8991	WiFi and supply-voltage watchdog. Shuts the Pi down cleanly on sustained low voltage or lost network.
44helper	`44helper`	9997	Optional. 44Net Connect, firewall, router and SvxLink helper.

The Pi Zero 2 W build

SysMon and the Dashboard come in two builds. The **Pi02w fork** is the lighter one for the 512 MB Pi Zero 2 W. Its files have `_pi02w_` in the name and its `VERSION` ends in `-pi02w`.

DASHBOARD · PI02W

Keeps every tab, including Phone and all digital modes.

- Searches the AllStar and EchoLink lists from disk instead of memory.
- Polls Asterisk and refreshes the page less often.

SYSMON · PI02W

Drops the heaviest tabs:

- Console, Net, Reg, Firewall and Zello.
- Everything else matches the full build.

instmon, wifimon, 44helper and the installer are **common**: one build for every Pi.

NOTE The Watchdog script (`asl_dvs_watchdog`) is not needed. The Dashboard's own service already restarts it if it stops, and the installer removes an old Watchdog if it finds one.

Before you start

What the Pi needs

- A Raspberry Pi Zero 2 W already running **AllStarLink** with **DVSwitch** (Analog_Bridge and MMDVM_Bridge) set up.
- `python3` and systemd. Both come with Raspberry Pi OS.
- A network connection, and SSH access with `sudo`.

Set a root password

The web pages ask for a login, and they check it against the Pi's **root** password. Raspberry Pi OS ships with root locked, so set a password once:

```
sudo passwd root
```

Free up memory first

A Pi Zero 2 W has about 415 MB usable. Check what is free before you add tools:

```
free -m
```

Services you don't use are the easiest memory to win back. Skip any you do use:

SERVICE	TYPICAL USE	TURN OFF WITH
PackageKit	~12 MB swap	`sudo systemctl mask --now packagekit`
P25Gateway	~22 MB swap	`sudo systemctl disable --now p25gateway`
Allmon3	~44 MB swap	`sudo systemctl disable --now allmon3`

Figures from a working Pi Zero 2 W node. PackageKit only serves Cockpit's "Software updates" page; `apt` works without it. Undo with `sudo systemctl enable --now <name>` (or `unmask` for packagekit).

Get the files and install instmon

Get the files

Put all the files in one folder on the Pi, for example your home folder:

```
# files from the main branch on GitHub
cd ~
B=https://raw.githubusercontent.com/kd8pgk/ASL-DVS/main
wget $B/instmon_v2_40_0.py
wget $B/install_v6_7_0.sh
wget $B/dashboard_pi02w_v9_4_0.py
wget $B/sysmon_pi02w_v6_13_67_11_20261011.py
wget $B/wifimon_v5_28_20261007.py
# optional
wget $B/asl_dvs_m17_44helper_v0_0_179_20261011.py
```

TIP Newer files may be on GitHub by the time you read this. Check the repo's file list and use the newest `_pi02w_` SysMon and Dashboard files.

instmon goes first. Once it is running, it can install and update everything else from the browser, and it updates itself.

STEP 1 · INSTMON

```
sudo python3 instmon_v2_40_0.py --install
```

Then open `http://<pi-address>:8990` and log in as **root**.

What `--install` does

- Copies the file into place and writes `instmon.service`.
- Writes the shared launcher `/usr/local/bin/asl_dvs_launch.py` (see *Memory*, section 8).
- Enables the service so it starts on boot, and starts it now.

Optional: stage the other files in instmon

Instead of running the installer by hand (Step 2), you can upload the files on instmon's page, or let **Check GitHub** download them into instmon's library, and press **Install** there. The installer also finds files staged in instmon's library.

Install the rest with the installer

STEP 2 · INSTALLER

```
sudo bash install_v6_7_0.sh
```

What it asks

1. **Operation.** If tools are already installed: `i` install / reinstall, `u` update binaries only, or `q` quit. Config is kept either way.
2. **Version.** If more than one version of a tool is found, pick one (Enter takes the newest).
3. **First-run identity.** On a fresh node: your callsign, ASL node number and a display label. They go into `/etc/asl_dvs/asl_dvs.conf`.

Where it looks for files

The installer's own folder, the current folder, `/root`, `/home/pi`, your home folder, `/tmp` and instmon's library. Any subset of the tools can be installed: it installs what it finds.

Which build it picks

For SysMon and the Dashboard the build is chosen first, then only that build's files are offered:

1. `SYSMON_VARIANT` / `DASH_VARIANT` = `pi02w` or `full`, if set;
2. otherwise the build already installed;
3. otherwise **pi02w on a Pi Zero 2 W**, full on any other Pi.

```
# force the Pi02w build (sudo needs the variables on its own line)
sudo SYSMON_VARIANT=pi02w DASH_VARIANT=pi02w bash install_v6_7_0.sh
```

CAREFUL If no file of the chosen build is found, that tool is skipped with a warning. The other build is never used in its place.

Without questions

Add `--non-interactive` (or `-y`). It picks the newest files, updates an existing install, and takes the identity from environment variables on a fresh node:

```
sudo AUTO_CALLSIGN=KD8PGK AUTO_NODE=652701 AUTO_LABEL="Travel Node" \
     bash install_v6_7_0.sh --non-interactive
```

When it finishes, the installer checks that each service is running and listening on its port.

5 · STEP 3

Open the pages and log in

PAGE	ADDRESS
Dashboard	`http://<pi-address>:8989`
SysMon	`http://<pi-address>:9999`
instmon	`http://<pi-address>:8990`
wifimon	`http://<pi-address>:8991`
44helper	`http://<pi-address>:9997`

Log in as **root** with the root password from section 2. The pages use plain HTTP, so only open them on your own network or over a VPN.

TIP On the first visit to the Dashboard, check that the callsign and node number in the header are right.

6 · STEP 4

Set up wifimon

On a first install, wifimon runs with both **watchdogs off**: it shows the WiFi and the voltage, but doesn't reconnect or shut the node down. When you log in to its page (port 8991) it asks *"Turn the watchdogs on?"*. Add your WiFi networks first, then turn them on there or in the **Watchdog** card:

- **WiFi watchdog:** reconnects to saved networks, and shuts the node down after 3 minutes with no connection. It waits, with no timer, until at least one network is saved.
- **Voltage watchdog:** shuts the node down after 30 seconds of low voltage.

Updates and reinstalls keep the watchdogs as they were. Add the networks it may reconnect to:

```
sudo python3 /usr/local/bin/wifimon.py --setup-wifi
```

You can also manage networks and each WiFi adapter on wifimon's page (port 8991).

CAREFUL wifimon shuts the Pi down cleanly on sustained low voltage or a sustained loss of network. Use a good power supply and cable: the Pi Zero 2 W is sensitive to voltage drop.

Updating from GitHub with instmon

The **GitHub Updates** card in instmon reads the repo's file list directly.

CHECK GITHUB

Shows the newest GitHub copy of each tool and whether it is newer than yours. Full and Pi02w builds get separate rows; rows for the build this Pi isn't using are tagged *other build*.

UPDATE

Downloads one file into instmon's library and checks it first: the same file GitHub listed, a version line, the right first line, a syntax check and `--install` support. Nothing installs until you press **Install**.

FULL UPDATE

1. Downloads and checks every new file before anything changes.
2. Pauses the web tools.
3. One tool at a time: saves a copy of the installed file, uninstalls it, installs the new one. If the new one fails, it puts the old one back and moves on.
4. Updates instmon last. A timer reinstalls the old instmon if the new one isn't answering after 90 seconds.
5. Ends with a summary.

The Pi Zero 2 W build choice

On a Pi Zero 2 W the GitHub Updates card shows a **Pi Zero 2 W build** choice: **Pi02w fork (default)** or **Full build (override)**. Full Update installs the chosen build, and switches an installed tool that doesn't match (listed as **SWITCH**).

Quiet System

Quiet System / Restore pauses SysMon, the Dashboard and 44helper before a big upload or install. Asterisk, the bridges, Allmon3, SSH and wifimon keep running. Everything comes back by itself after 30 minutes.

Memory on a 512 MB Pi

The launcher

The Pi02w SysMon and Dashboard, instmon, wifimon and 44helper all start through `/usr/local/bin/asl_dvs_launch.py`:

```
ExecStart=/usr/bin/python3 /usr/local/bin/asl_dvs_launch.py /usr/local/bin/sysmon.py
```

Started directly, Python compiles the whole file on every start and keeps that memory. The launcher loads the tool as a module instead, so Python saves the compiled copy in `__pycache__` and reuses it. That is about half the memory, and twice as fast to start.

Three more memory savers

Launcher version 2 also turns these on. Each has an off switch:

SAVER	WHAT IT DOES	OFF SWITCH
-OO	Drops the built-in help text (docstrings) from the loaded code.	`/etc/asl_dvs/launch_no_optimize`
Pool cap	`MALLOC_ARENA_MAX=2`: at most 2 memory pools instead of up to 8 per CPU core.	`/etc/asl_dvs/launch_no_arena_cap`
Trim	1 minute after start, then every 5 minutes, hands freed memory back to Linux.	`/etc/asl_dvs/launch_no_trim`

To turn one off, create its file and restart the services. Delete the file and restart to turn it back on:

```
sudo touch /etc/asl_dvs/launch_no_arena_cap
sudo systemctl restart sysmon asl_dvs_dashboard instmon wifimon
```

NOTE The first start after an update compiles everything once more, so memory runs high. Measure after the next restart. With the first launcher version one node fell from about 390 to 294 of 415 MiB; the three savers haven't been measured on a Pi yet.

Checking memory

The Pi Zero 2 W uses zram: swap that is compressed in RAM. A program's real size is its RAM *plus* its swap:

```
for p in $(pgrep -f asl_dvs_launch.py); do
  printf "%-40s %s\n" "$(tr '\0' ' ' </proc/$p/cmdline | awk '{print $NF}')" \
    "$(grep -E 'VmRSS|VmSwap' /proc/$p/status | tr -s ' \n' ' ')"
done; free -m
```

Biggest swap users across the whole Pi:

```
for f in /proc/[0-9]*/status; do awk '/^Name:/{n=$2} /^VmSwap:/{if($2>0) printf "%7.1f MB  %s
\n",$2/1024,n}' $f 2>/dev/null; done | sort -rn | head -10
```

Troubleshooting

A page doesn't load

```
# is the service running?
systemctl status sysmon --no-pager
# its last 30 log lines
journalctl -u sysmon -n 30 --no-pager
# is anything listening on the port?
ss -tlnp | grep 9999
```

Use the service name and port of the tool in question (table in section 1).

The login is refused

The pages check the **root** password. Set or reset it with `sudo passwd root`.

The installer says "No pi02w … build found"

The `_pi02w_` file for that tool isn't in any of the folders it searches. Put it next to the installer and run it again.

The Pi is slow or runs out of memory

- Run the memory commands in section 8 and look for the biggest swap users.
- Check that every tool line shows `asl_dvs_launch.py`. A tool started as plain `python3 tool.py` uses about twice the memory: run its `--install` again.
- Close Cockpit's page when you're done: its two `cockpit-bridge` processes use about 27 MB while it is open.

A tool misbehaves after an update

Turn the memory savers off one at a time (section 8), starting with the pool cap, and restart. If that doesn't help, reinstall the previous version from instmon's library.

SysMon's own checks

```
sudo python3 /usr/local/bin/sysmon.py --selftest
```

It runs SysMon's security checks against this Pi and prints PASS / WARN / FAIL for each, without starting the web page.

10 · REMOVING

Removing the tools

Remove one tool with its own uninstall:

```
sudo python3 /usr/local/bin/sysmon.py --uninstall
```

Remove every tool except instmon:

```
sudo bash uninstall_all_v1_4_0.sh
```

This leaves `/etc/asl_dvs/`, instmon's library and `/etc/wifimon/wifimon.conf` in place. Remove instmon last with its own `--uninstall`. The launcher is removed once no service uses it.

11 · QUICK REFERENCE

Quick reference

TASK	COMMAND
Install instmon	`sudo python3 instmon_v2_40_0.py --install`
Install the rest	`sudo bash install_v6_7_0.sh`
Set the login password	`sudo passwd root`
Add WiFi networks	`sudo python3 /usr/local/bin/wifimon.py --setup-wifi`
Restart the web tools	`sudo systemctl restart sysmon asl_dvs_dashboard instmon wifimon`
A tool's log	`journalctl -u <service> -n 30 --no-pager`
Memory overview	`free -m` · `zramctl`
Saver off switch	`sudo touch /etc/asl_dvs/launch_no_<optimize|arena_cap|trim>`
Remove all but instmon	`sudo bash uninstall_all_v1_4_0.sh`

SERVICE	PORT	CONFIG
asl_dvs_dashboard	8989	`/etc/asl_dvs/asl_dvs.conf`
sysmon	9999	`/etc/sysmon/sysmon.conf`
instmon	8990	`/etc/asl_dvs/` · library in `/etc/asl_dvs/instmon_library`
wifimon	8991	`/etc/wifimon/wifimon.conf`
44helper	9997	`/etc/44helper/44helper.conf`

Phone and its dialplan

The Dashboard's **Phone** tab places and answers calls through Asterisk, the phone engine under AllStarLink. Nothing changes in Asterisk until you tick **Activate** on the Edit page (Phone section). After that, every **Save Phone** rewrites the Dashboard's own files and reloads Asterisk.

What Activate adds

FILE IN /ETC/ASTERISK	WHAT THE DASHBOARD ADDS
`rpt.conf`	One private node per phone network (1000–1999), between its `dvs-phone` marker lines.
`extensions.conf`	One line at the end: `#tryinclude dvs_phone_extensions.conf` (the phone dialplan).
`pjsip.conf` / `iax.conf`	Include lines for the SIP and IAX2 account files.
`modules.conf`	Load lines for the phone modules. New SIP modules need one Asterisk restart.
`/var/lib/asterisk`	Three small scripts: hang up, radio tone codes, radio dial requests.

From the radio

CODE	DOES
`*61` + number	Dial. The Dashboard places the call (the Phone tab must be open).
`*62` or `*65`	Hang up.
`*63` / `*64`	Phone patch on / off.
`*980` `*981` `*982`	Send *, # or *99 into the call.

How calls reach the radio

Each phone network has its own node, linked to your radio node only while the Phone tab is open. Calls join that node with voice keying (`rpt(node,Pv)`), so a simplex radio transmits only while the far end talks and can hear you in the gaps. Tick **Listen only** to link the phone node one-way: you hear calls, and nothing from the radio goes into them.

```
# is the phone node linked, and which way? (R = listen only, T = two-way)
sudo asterisk -rx "rpt xnode <your-node>" | grep ALINKS
# the dialing rules Asterisk has loaded for one network
sudo asterisk -rx "dialplan show dvs-net-<network-id>"
```

TIP SysMon's **Phone** tab reads all of this from the files, read-only, and checks it: nodes, networks, sign-in, dialing rules and the modules the phone needs.

NOTE The nuts and bolts (dialplan syntax, patterns, every context the Dashboard writes, autopatch, call flows and troubleshooting) are in `docs/ASL-DVS_Pi02w_Phone_Dialplan_Guide.pdf` .